Student Information

Information	
Name	Elyssa Olivares
Section	BSCS 4B
Assigned Dataset	Animal Image Dataset (90 Different Animals)
Kaggle Dataset URL	https://www.kaggle.com/datasets/iamsouravbanerjee/animal-image-dataset-90-different-animals/data?utm_source=chatgpt.com
Date Submitted	August 28, 2026

Part 1 — Dataset Exploration

1.1 Dataset Description

Brief Description

Describe your assigned dataset in your own words.

The dataset assigned for this activity is the **Animal Image Dataset (90 Different Animals)** that was curated by Sourav Banerjee on Kaggle. It is a multi-class dataset that is mainly used for computer vision containing approximately 5,400 color RGB images distributed across 90 distinct animal categories. It also features a balanced distribution with 60 high-quality images per class, which minimizes severe class imbalance issues during model training. The images gathered to create this dataset are also from Google Images.

1.2 Dataset Characteristics

Complete the following table based on your actual dataset.

Dataset Characteristic	Result
Number of Classes	90
Total Number of Images	5,400 images (approx. 60 per class)
Image Format	JPG / JPEG / PNG
Image Dimensions	Varied raw dimensions (resized to 32x32, 64x64, or 128x128 for training)
Color Channels	3 (RGB)
Smallest Class	60 images (balanced)
Largest Class	60 images (balanced)

1.3 Classes

List all classes found in your dataset.

#	Class Name	Number of Images	11	caterpillar	60
1	antelope	60	12	chimpanzee	60
2	badger	60	13	cockroach	60
3	bat	60	14	cow	60
4	bear	60	15	coyote	60
5	bee	60	16	crab	60
6	beetle	60	17	crow	60
7	bison	60	18	deer	60
8	boar	60	19	dog	60
9	butterfly	60	20	dolphin	60
10	cat	60	21	donkey	60

22	dragonfly	60	46	leopard	60
23	duck	60	47	lion	60
24	eagle	60	48	lizard	60
25	elephant	60	49	lobster	60
26	flamingo	60	50	mosquito	60
27	fly	60	51	moth	60
28	fox	60	52	mouse	60
29	goat	60	53	octopus	60
30	goldfish	60	54	okapi	60
31	goose	60	55	orangutan	60
32	gorilla	60	56	otter	60
33	grasshopper	60	57	owl	60
34	hamster	60	58	ox	60
35	hare	60	59	oyster	60
36	hedgehog	60	60	panda	60
37	hippopotamus	60	61	parrot	60
38	hornbill	60	62	pelecaniformes	60
39	horse	60	63	penguin	60
40	hummingbird	60	64	pig	60
41	hyena	60	65	pigeon	60
42	jellyfish	60	66	porcupine	60
43	kangaroo	60	67	possum	60
44	koala	60	68	raccoon	60
45	ladybugs	60	69	rat	60

70	reindeer	60
71	rhinoceros	60
72	sandpiper	60
73	seahorse	60
74	seal	60
75	shark	60
76	sheep	60
77	snake	60
78	sparrow	60
79	squid	60
80	squirrel	60
81	starfish	60
82	swan	60
83	tiger	60
84	turkey	60
85	turtle	60
86	whale	60
87	wolf	60
88	wombat	60
89	woodpecker	60
90	zebra	60

1.4 Initial Observation

Based on your initial exploration, what did you observe about the dataset?

> Upon exploring the dataset, I first noticed the number of classes which is too big to look at but the good thing was, every class has 60 images that make them balanced and surprisingly clean. The classes are also wide in a way that it includes wild mammals, domestic pets, insects, and even marine animals. It was already organized based on class and alphabetically, that makes it easier for training a model. Because the images were curated directly from Google Images, they exhibit a high degree of real-world variability in lighting, background environments, image resolutions, and camera angles. Looking at the images, I noticed that their sizes and file extensions are different so it made me think that during model training the dataset requires standard image preprocessing, resizing, normalization, and such.

Part 2 — Train, Validation, and Test Sets

2.1 Dataset Splitting

Describe how you divided your dataset into:

- Training set
- Validation set
- Testing set

Dataset Split

Dataset	Number of Images	Percentage
Training	4,320	80%
Validation	536	9.9%
Testing	544	10.1%
Total	5,400	100%

2.2 Purpose of Each Dataset

Explain the purpose of each set.

Training Set: The training set is used for teaching the model to adjust weights based on the number of training images (4,320) so it can learn the patterns that distinguish or identify each animal class.

Validation Set: The validation set is used during training to monitor the performance of unseen data every after epoch, it guides decisions like early stopping and learning-rate reduction. This happens without being used to update the model weights directly.

Testing Set: The testing set is used after training and it will only happen once when it (training) is finished. This is to give an unbiased estimate of how well the final model generalizes or adjusts to the new images.

2.3 Observation

Why is it important to keep the test dataset separate from the training dataset?

> It is important to keep the dataset separate from the training dataset to make the model's output accurate and unbiased. If the dataset used for testing was seen during training, the model could simply memorize the images making the result close to perfect accuracy and not on the way it must perform looking into newly fed images.

Part 3 — Simple Neural Network

3.1 Model Architecture

Present the architecture of your neural network.

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 12288)	0
dense (Dense)	(None, 512)	6,291,968
batch_normalization (BatchNormalization)	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 90)	23,130

Total params: 6,449,498 (24.60 MB)
Trainable params: 6,447,962 (24.60 MB)
Non-trainable params: 1,536 (6.00 KB)

3.2 Model Configuration

Complete the table.

Component	Configuration
Input Image Size	64 x 64 x 3
Flatten	Converts input image into 1D vector of 12,288 values
Dense Units	512 -> 256 -> 90 (output)
Activation Function	ReLu (hidden layer) and Softmax (output layer)
Optimizer	Adam with learning rate of 0.001
Loss Function	Sparse Categorical Crossentropy
Number of Epochs	40 (stopped at 26 triggered by EarlyStopping)
Batch Size	32

3.3 Understanding Flatten

Explain what happens when an image passes through the `Flatten` layer.

The flatten layer takes the 3 dimensional image array the height x width x color channels, which the model here used 64 x 64 x 3 and reshapes it to a single or 1 dimensional vector of 12, 288 numbers, with no learnable parameters of its own. Here, every pixel becomes one independent input feature and means that the network loses all information about which pixels were spatially near each other. CNN uses the concept of edges, shapes, or textures.

Part 4 — Model Training

4.1 Training Configuration

Record your training configuration.

Parameter	Value
Epochs	40 but it stopped early in 26 via EarlyStopping
Batch Size	32
Optimizer	Adam
Learning Rate	started to 0.001, reduced by ReduceLROnPlateau to 0.0000625 by epoch 24
Training Images	4,320
Validation Images	536

4.1 Training and Validation Curves

Figure 3. Training and Validation Loss

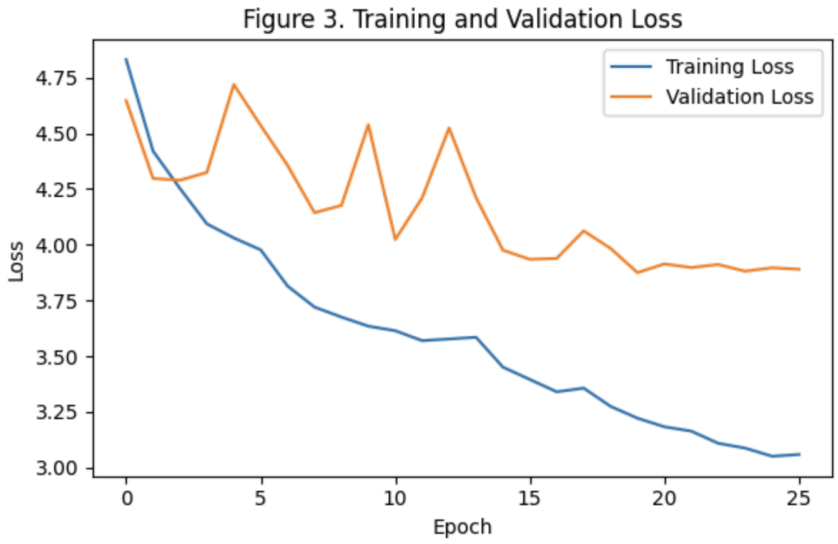
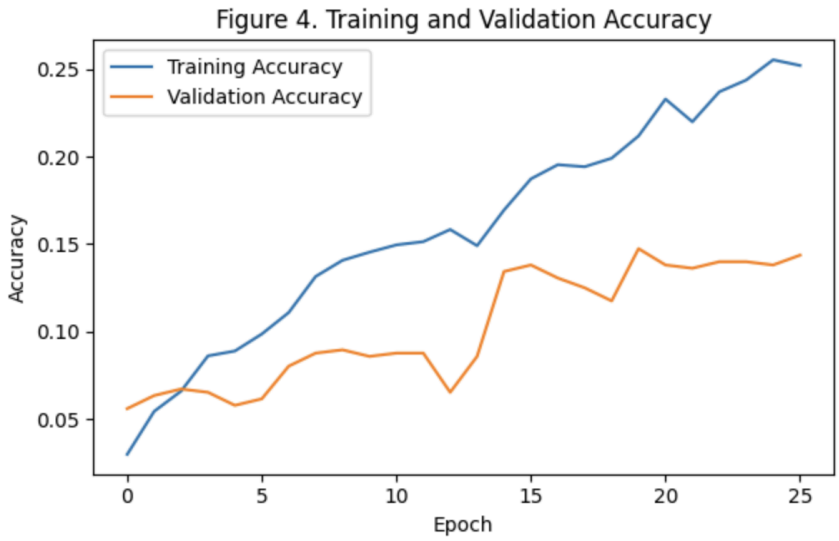


Figure 4. Training and Validation Accuracy



4.2 Interpretation

What do the training curves tell you about your model?

> The training curves show that the model was steadily learning, training loss fell from 4.83 to about 3.05 and training accuracy rose from 3% to roughly 25% over epochs. You can see the improvements in the overall accuracy from about 6% to 14% but far more slowly and noisily than training accuracy, and validation loss stayed higher than training loss throughout. That gap indicates that the model wasn't learning well since the accuracy was low even on the training set and which points to underfitting caused by Flatten and Dense. It is the architecture's inability to capture spatial patterns in the images, rather than the model simply memorizing the data.

Part 5 — Experiment: Image Resolution

5.1 Experiment Objective

The objective of this experiment is to determine:

> **Does image resolution affect the performance of a simple neural network?**

The same model and dataset will be used while changing only the image resolution.

5.2 Experimental Conditions

Experiment	Image Resolution	Input Features
Experiment 1	32 × 32 × 3	3,072
Experiment 2	64 × 64 × 3	12,288
Experiment 3	128 × 128 × 3	49,152

Calculate the number of input features for each resolution.

Part 6 — Experiment Results

6.1 Performance Comparison

Complete the following table.

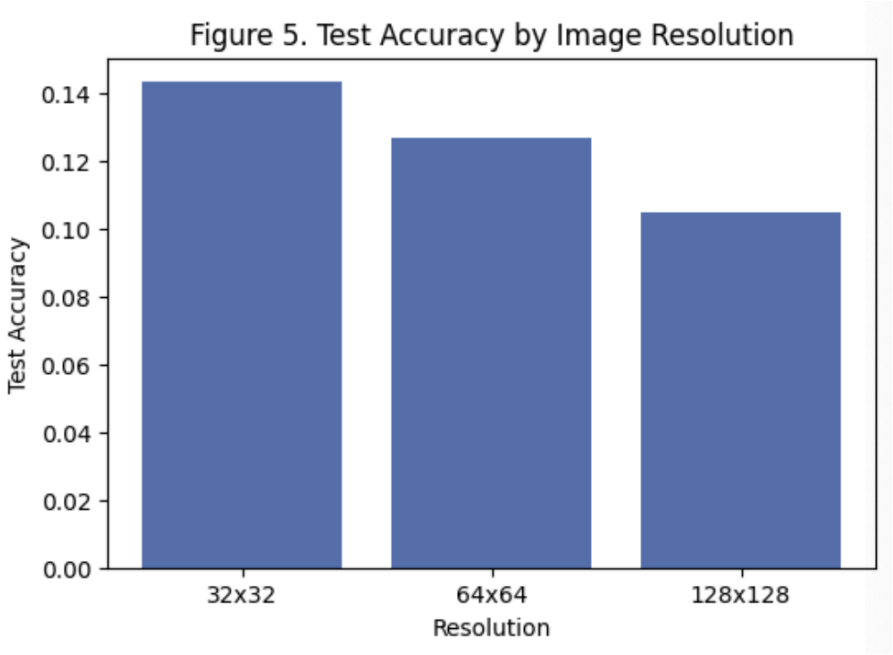
Resolution	Input Features	Parameters	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time
32 × 32	3072	1730906	0.219907	0.121269	0.143382	135.501324
64 × 64	12288	6449498	0.198611	0.113806	0.126838	293.397155
128 × 128	49152	25323866	0.244676	0.147388	0.104779	1749.077757

	Resolution	Input Features	Parameters	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time (s)
0	32x32	3072	1730906	0.219907	0.121269	0.143382	135.501324
1	64x64	12288	6449498	0.198611	0.113806	0.126838	293.397155
2	128x128	49152	25323866	0.244676	0.147388	0.104779	1749.077757

6.2 Accuracy Comparison

Create a graph comparing the test accuracy of the three experiments.

Figure 5. Test Accuracy by Image Resolution



6.3 Training Time Comparison

Create a graph comparing the training time of the three experiments.

Figure 6. Training Time by Image Resolution



6.4 Experimental Observation

Describe what happened when the image resolution increased.

> When the image resolution gets higher, the test accuracy becomes low while in terms of training time, the higher the resolution it's faster. The training time increased because the GPU must process vastly larger feature maps at every layer. Simultaneously, the test accuracy drops because standard convolutional layers trained on smaller patches fail to capture the global structure when the features stretch out. At 32x32, a standard 3×3 filter can see a massive percentage of the object, but at 128x128, that exact same filter only sees a tiny, unidentifiable texture fragment, causing the network to lose context and overfit to fine background noise.

Part 7 — Model Evaluation

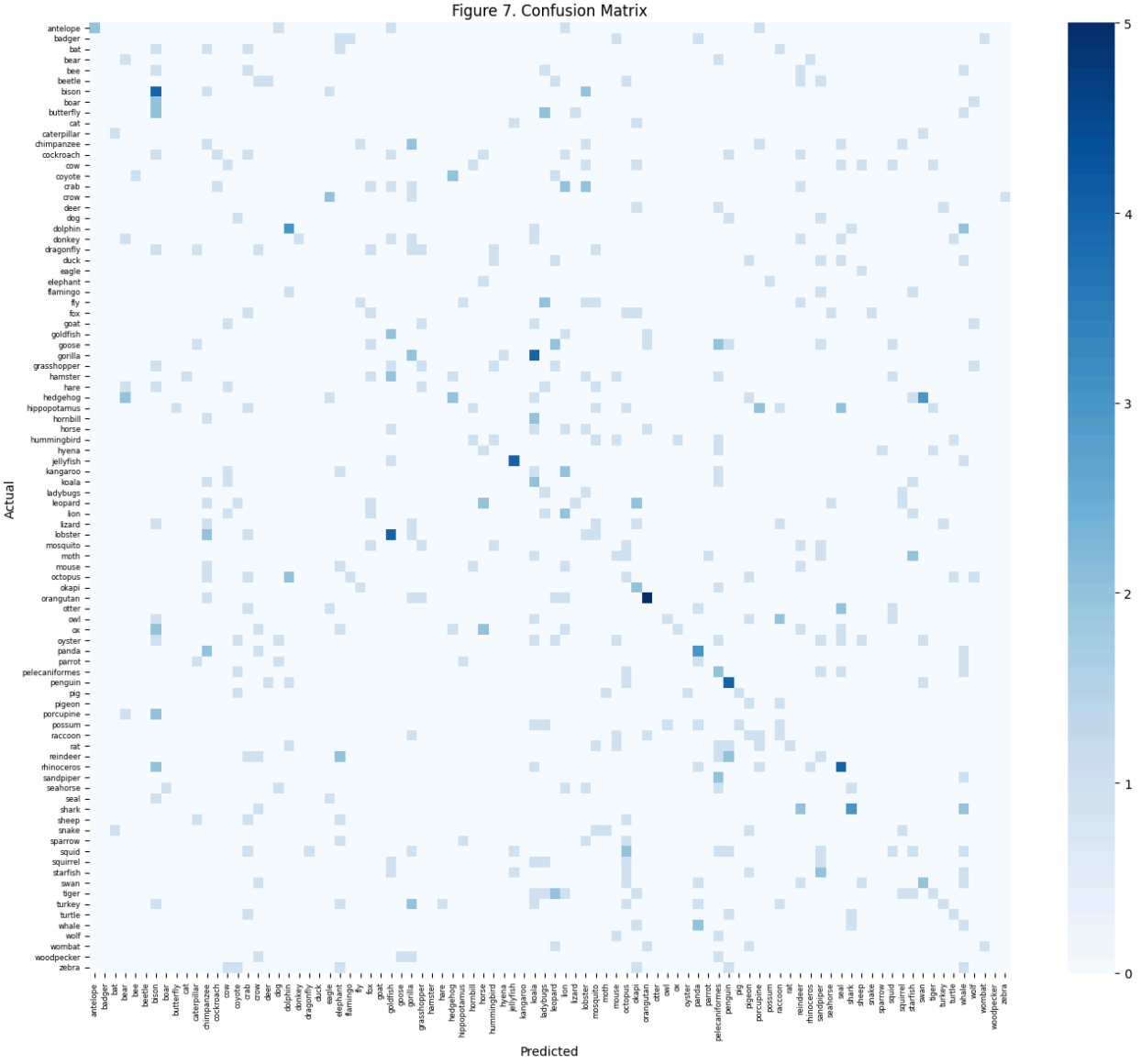
7.1 Test Performance

Report the performance of your final/best model.

Metric	Result
Test Loss	3.8835
Test Accuracy	12.68%

7.2 Confusion Matrix

Figure 7. Confusion Matrix



7.3 Classification Performance

Insert your classification report. (*you can see the full output to the last page*)

	precision	recall	f1-score	support
antelope	1.000000	0.333333	0.500000	6.000000
badger	0.000000	0.000000	0.000000	5.000000
bat	0.000000	0.000000	0.000000	5.000000
bear	0.166667	0.333333	0.222222	3.000000
bee	0.000000	0.000000	0.000000	5.000000
...	...	...	...	...
woodpecker	0.000000	0.000000	0.000000	4.000000
zebra	0.000000	0.000000	0.000000	6.000000
accuracy	0.126838	0.126838	0.126838	0.126838
macro avg	0.140246	0.119851	0.103549	544.000000
weighted avg	0.168627	0.126838	0.117543	544.000000

93 rows × 4 columns

7.4 Class Performance

Best Performing Class

Class: Jellyfish

Reason/Observation:

The jellyfish class attained an F1-Score = 0.615 it is I think due to its visually distinctive color, translucency, and shape compared to most other animals, so even the model is spatially blinded (due to Flatten+Dense) it can still pick up on consistent color/brightness patterns across pixels.

Most Difficult Class

(There are actually a lot of difficult classes but I will just state the most difficult among them)

Class: Badger

Reason/Observation:

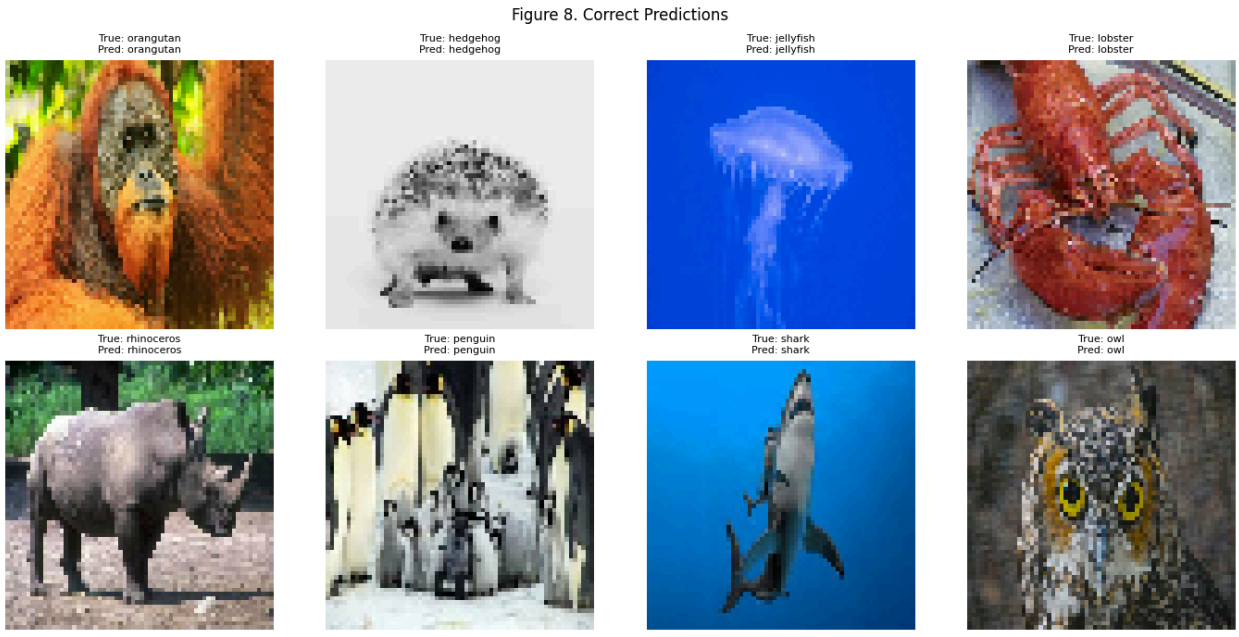
The Badger class resulted in F1-Score = 0.000. This means that the model wasn't able to identify a single badger. Looking at the images, badgers likely share overlapping fur colors/textures and body shapes with other similar sized mammal classes from the dataset. Since Flatten layers discard spatial or shape information entirely, the model has no way to use silhouette or structural causes to tell them apart.

Part 8 — Prediction Analysis

8.1 Correct Predictions

Display examples of correctly classified images.

Figure 8. Correct Predictions

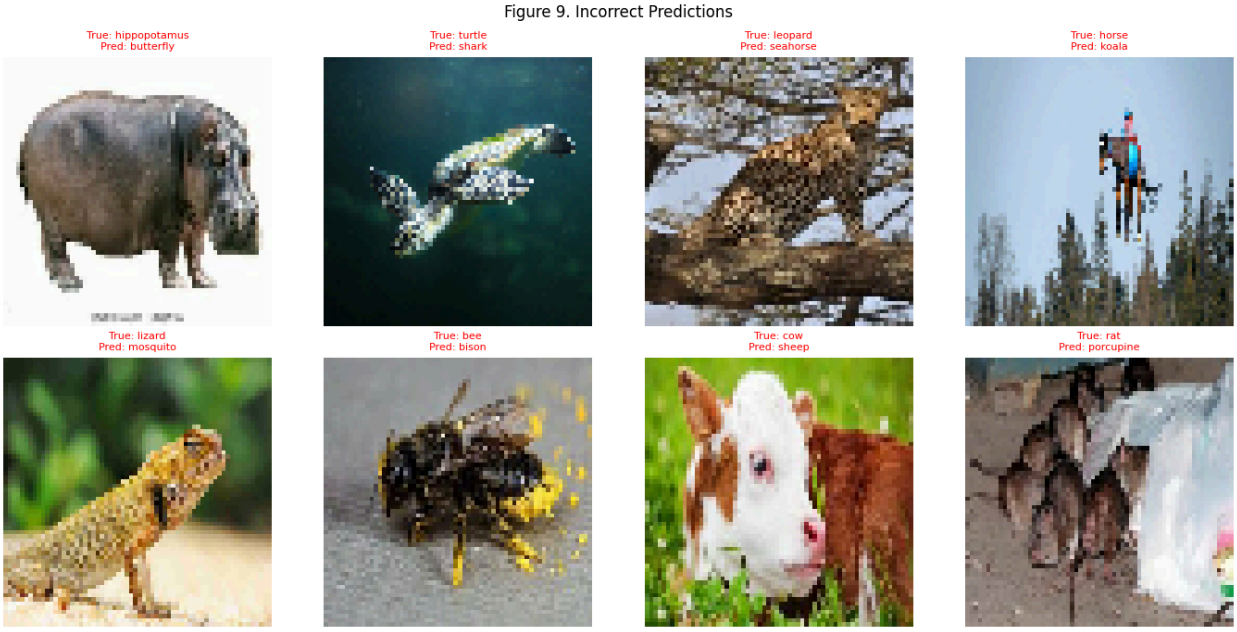


They are very similar in terms of their shapes and color depth. For example, the rhino has a unique horn shape and its body structure also helps the model to identify it correctly, similar to the shark, lobster, and many more.

8.2 Incorrect Predictions

Display examples of incorrectly classified images.

Figure 9. Incorrect Predictions



8.3 Prediction Errors

Choose at least **three incorrect predictions** and analyze them.

Image	Actual Class	Predicted Class	Possible Reason
1	Turtle	Shark	They are likely similar in terms of color highlights and both are under water. The angle of the turtle here was somehow elongated and can be easily mistook as a shark.

2	Hippopotamus	Butterfly	The width and edges of both animals are somehow the same, also the dark color of the hippo affects the model's identification.
3	Cow	Sheep	Since the resolution is low, the edges of the cow's image cannot be clearly seen so it can be mistakenly identified as a sheep due to smooth edges and of course the color isn't that far.

Discussion

Why do you think the neural network confused these classes?

> The Flatten layer treats each pixel as an independent input without any spatial meaning. The network can only learn coarse or rough statistical patterns like overall color distribution and brightness, but cannot recognize shapes, edges, or textures like a convolutional network can. Classes similar in color, background/habitat, or overall body proportions are thus easily confused, as the model has no mechanism to focus on the distinguishing structural details a human (or a CNN) would use to tell them apart.

Part 9 — Reflection

9.1 What Did You Observe?

This activity demonstrated that a basic Flatten+Dense neural net can learn something from raw pixels, training accuracy steadily increased from near-random (1.1% for 90 classes) to ~25% over 26 epochs, more than 20x better than chance. But it didn't generalize well, the validation accuracy only got to 14% and the test accuracy only got up to 12.68%, less than half of the training accuracy. Validation loss was noisy and high (3.9–4.7) throughout, never catching up with training loss (which dropped to 3.05), a classic sign of limited generalization.

The reason is tied directly to the architecture Flatten turns each image into an unordered vector of pixel values, throwing away all spatial relationships between

neighboring pixels – the very cues (edges, shapes, textures) that animal classification relies on. We see this in the per-class results as well: jellyfish did best (F1 = 0.615), likely due to the characteristic color/translucency that survives flattening, and badger did worst (F1 = 0.000), likely confused with visually similar mammals since the model cannot use shape to distinguish them.

Part 10 — Conclusion

Write a concise conclusion summarizing:

1. The dataset you used.
2. The performance of your simple neural network.
3. The effect of image resolution.
4. The major problems or limitations encountered.
5. Your overall answer to the question:

Can a Simple Neural Network Recognize Images?

Conclusion:

The Animal Image Dataset (90 Different Animals) from Kaggle contains 90 animal classes with 60 images per class, so this dataset is perfectly balanced with 5,400 images. The images are with different resolutions, aspect ratios, backgrounds, so it was resized and normalized before training. The dataset was split into 4320 training images (80%), 536 validation images (9.9%) and 544 test images (10.1%) with the test set held separate from training. An easy neural network with a Flatten layer, two Dense layers (512, 256 units) and a 90-class Softmax output was able to learn patterns from the images but performance was limited. The training accuracy increased from 3% to 25.5% and the training loss decreased from 4.83 to 3.05 over 26 epochs. However, the validation accuracy was only about 14%, and final test accuracy was 12.68%, compared to a 1.1% random-guessing baseline. The wide difference between training accuracy and validation accuracy is a sign of poor generalization, which likely means the model learned some patterns of the training images but not features that can reliably distinguish unseen animals images.

The image resolution presents a trade-off for this type of network. Increasing image resolution from 32×32 to 64×64 to 128×128 greatly increases the number of input features and model parameters. While higher image resolution preserves visual information it also makes the model more computationally expensive and increases the risk of overfitting. Since the Flatten

layer does not preserve relationships the additional pixel information may provide limited benefits. Therefore increasing image resolution is unlikely to solve the model's main weaknesses. The biggest limitation was the Flatten plus architecture. Flattening the image removes the relationships between neighboring pixels making it difficult for the network to recognize important visual features such as edges, shapes, textures and contours. This was reflected in the class results: jellyfish performed best with an F1-score of 0.615 likely because of its visual characteristics while badger had an F1-score of 0.000 meaning it was never correctly classified. The 90-class classification task was also challenging because many animals have similar features.

Can a Simple Neural Network Recognize Images?The overall answer is: only **partially**. A simple Flatten+Dense neural network is capable of learning to recognize images better than random chance. In this case, over 12x better, by picking up on coarse, non-spatial patterns such as color and brightness distributions. However, it is fundamentally limited for real-world image recognition tasks like this one, since it has no mechanism to represent the spatial structure that most visual distinctions actually depend on. This is precisely the gap that convolutional neural networks were designed to close, by preserving and exploiting local pixel neighborhoods through convolution and pooling operations, CNNs can learn the shape- and texture-based features that a Flatten-based network cannot, which is why they consistently and substantially outperform simple dense networks on image classification tasks.

Part 11 — Appendix

Include the following:

Appendix A — Google Colab Notebook

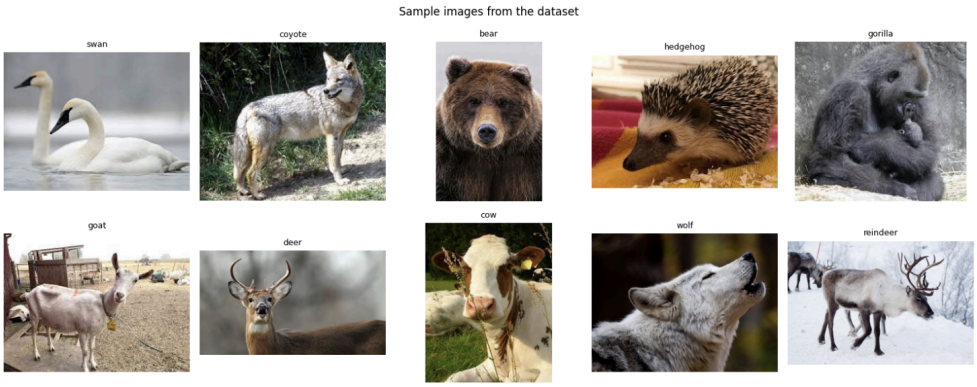
Provide the link to your completed Google Colab notebook.

Colab Link: L2_Olivares.ipynb or https://colab.research.google.com/drive/1ua2vfLg7QcGWDlFBpUbwobUEyDG6Yd-x?usp=sharing

Appendix B — Additional Results

Include any additional graphs, tables, screenshots, or results that support your discussion.

Sample Images from the Dataset Labeled with its Class



Additional Incorrect Class Predictions

	Image Index	Actual Class	Predicted Class
1	15	dog	penguin
2	404	lizard	turkey
3	510	possum	pig
4	33	caterpillar	bat
5	145	ox	bison

Full 93 Classes' Precision, Recall, F1-Score, and Support

1 to 25 of 93 entries Filter

index	precision	recall	f1-score	support
antelope	1.0	0.3333333333333333	0.5	6.0
badger	0.0	0.0	0.0	5.0
bat	0.0	0.0	0.0	5.0
bear	0.16666666666666666	0.3333333333333333	0.2222222222222222	3.0
bee	0.0	0.0	0.0	5.0
beetle	0.0	0.0	0.0	6.0
bison	0.16	0.5	0.24242424242424243	8.0
boar	0.0	0.0	0.0	3.0
butterfly	0.0	0.0	0.0	6.0
cat	0.0	0.0	0.0	2.0
caterpillar	0.0	0.0	0.0	2.0
chimpanzee	0.07142857142857142	0.125	0.09090909090909091	8.0
cockroach	0.5	0.125	0.2	8.0
cow	0.14285714285714285	0.125	0.13333333333333333	8.0
coyote	0.0	0.0	0.0	4.0
crab	0.0	0.0	0.0	9.0
crow	0.0	0.0	0.0	4.0
deer	0.0	0.0	0.0	3.0
dog	0.0	0.0	0.0	3.0
dolphin	0.375	0.42857142857142855	0.4	7.0
donkey	1.0	0.125	0.2222222222222222	8.0
dragonfly	0.0	0.0	0.0	8.0
duck	0.0	0.0	0.0	6.0
eagle	0.0	0.0	0.0	1.0
elephant	0.0	0.0	0.0	2.0

Show 25 per page 1 2 3 4

26 to 50 of 93 entries Filter

index	precision	recall	f1-score	support
flamingo	0.0	0.0	0.0	3.0
fly	0.3333333333333333	0.14285714285714285	0.2	7.0
fox	0.125	0.16666666666666666	0.14285714285714285	6.0
goat	0.0	0.0	0.0	4.0
goldfish	0.11764705882352941	0.5	0.19047619047619047	4.0
goose	0.0	0.0	0.0	10.0
gorilla	0.125	0.2857142857142857	0.17391304347826086	7.0
grasshopper	0.16666666666666666	0.16666666666666666	0.16666666666666666	6.0
hamster	0.0	0.0	0.0	9.0
hare	0.0	0.0	0.0	6.0
hedgehog	0.3333333333333333	0.1818181818181818	0.23529411764705882	11.0
hippopotamus	0.0	0.0	0.0	11.0
hornbill	0.0	0.0	0.0	3.0
horse	0.0	0.0	0.0	5.0
hummingbird	0.2	0.14285714285714285	0.16666666666666666	7.0
hyena	0.0	0.0	0.0	4.0
jellyfish	0.5714285714285714	0.6666666666666666	0.6153846153846154	6.0
kangaroo	0.0	0.0	0.0	6.0
koala	0.09090909090909091	0.3333333333333333	0.14285714285714285	6.0
ladybugs	0.09090909090909091	0.3333333333333333	0.14285714285714285	3.0
leopard	0.0	0.0	0.0	10.0
lion	0.13333333333333333	0.3333333333333333	0.19047619047619047	6.0
lizard	0.0	0.0	0.0	7.0
lobster	0.07692307692307693	0.1	0.08695652173913043	10.0
mosquito	0.0	0.0	0.0	6.0

Show 25 per page 1 2 3 4

51 to 75 of 93 entries Filter

index	precision	recall	f1-score	support
moth	0.0	0.0	0.0	7.0
mouse	0.0	0.0	0.0	5.0
octopus	0.0625	0.1111111111111111	0.08	9.0
okapi	0.16666666666666666	0.5	0.25	4.0
orangutan	0.5	0.5	0.5	10.0
otter	0.0	0.0	0.0	6.0
owl	0.5	0.14285714285714285	0.2222222222222222	7.0
ox	0.5	0.09090909090909091	0.15384615384615385	11.0
oyster	0.0	0.0	0.0	11.0
panda	0.25	0.42857142857142855	0.3157894736842105	7.0
parrot	0.0	0.0	0.0	5.0
pelecaniformes	0.09523809523809523	0.2857142857142857	0.14285714285714285	7.0
penguin	0.3076923076923077	0.5	0.38095238095238093	8.0
pig	0.5	0.25	0.3333333333333333	4.0
pigeon	0.1111111111111111	0.5	0.18181818181818182	2.0
porcupine	0.0	0.0	0.0	3.0
possum	0.0	0.0	0.0	6.0
raccoon	0.1111111111111111	0.16666666666666666	0.13333333333333333	6.0
rat	1.0	0.14285714285714285	0.25	7.0
reindeer	0.0	0.0	0.0	8.0
rhinoceros	0.5	0.1	0.16666666666666666	10.0
sandpiper	0.0	0.0	0.0	3.0
seahorse	0.0	0.0	0.0	6.0
seal	0.0	0.0	0.0	2.0
shark	0.42857142857142855	0.375	0.4	8.0

Show 25 per page 1 2 3 4

76 to 93 of 93 entries Filter

index	precision	recall	f1-score	support
sheep	0.0	0.0	0.0	4.0
snake	0.0	0.0	0.0	5.0
sparrow	0.0	0.0	0.0	4.0
squid	0.16666666666666666	0.08333333333333333	0.1111111111111111	12.0
squirrel	0.0	0.0	0.0	5.0
starfish	0.0	0.0	0.0	7.0
swan	0.25	0.25	0.25	8.0
tiger	0.25	0.1111111111111111	0.15384615384615385	9.0
turkey	0.3333333333333333	0.1	0.15384615384615385	10.0
turtle	0.25	0.25	0.25	4.0
whale	0.058823529411764705	0.2	0.09090909090909091	5.0
wolf	0.0	0.0	0.0	2.0
wombat	0.5	0.25	0.3333333333333333	4.0
woodpecker	0.0	0.0	0.0	4.0
zebra	0.0	0.0	0.0	6.0
accuracy	0.12683823529411764	0.12683823529411764	0.12683823529411764	0.12683823529411764
macro avg	0.1402461131823877	0.11985129068462402	0.10354868093307543	544.0
weighted avg	0.16862709753361785	0.12683823529411764	0.11754319305041386	544.0

Show 25 per page 1 2 3 4